

Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет
"Высшая школа экономики"»

Московский институт электроники и математики им.
А.Н. Тихонова НИУ ВШЭ
Департамент компьютерной инженерии

Курс: Алгоритмизация и программирование

ОТЧЕТ
по лабораторной работе №5

Раздел	Максимальная оценка	Итоговая оценка
Работа программы	1	
Тесты	1	
Правильность алгоритма	3	
Листинг	0,5	
Ответы на вопросы	2	
Дополнительное задание	3	
Итого		

Студент: Солодилов Михаил Анатольевич
Группа: БИВ253
Вариант: 232 (5, 12, 2)
Руководитель:
Оценка:
Дата сдачи:

Оглавление

Задание	2
Внешняя спецификация	3
Листинг программы	4
Распечатка тестов к программе и результатов	8

Задание

1. Создать файл для хранения действительных чисел, вводимых с клавиатуры. Прочитать этот файл и вычислить максимальное среди отрицательных чисел.
2. Создать связанный список для хранения целых чисел. Число записей неизвестно, но можно ввести число, являющееся признаком окончания ввода данных. Для четных номеров вариантов организовать очередь, а для нечетных – стек. Для исходного списка решить следующую задачу: вставить заданное число $A1$ после каждого элемента с нечетным номером.
3. Для полученного списка решить следующую задачу: упорядочить по убыванию методом «пузырька» элементы списка, расположенные после первого положительного элемента.

Внешняя спецификация

Лабораторная работа №5

Задание №1

Введите действительные числа, ввод закончите пустой строкой:
< x >

До пустой строки
При $max = -\infty$

В последовательности не было отрицательных чисел

Иначе

$max = \ll max \gg$

Задание №2

Введите целые числа, ввод закончите пустой строкой:
< n >

До пустой строки

Очередь:
 $\ll q[0] \gg, \ll q[1] \gg, \dots$

Введите A1:
< $A1$ >

Очередь:
 $\ll q[0] \gg, \ll q[1] \gg, \dots$

Задание №3

При $pos = NULL$

Нет положительных элементов

Иначе

Очередь:
 $\ll q[0] \gg, \ll q[1] \gg, \dots$

Листинг программы

```
1 #include <stdio.h>
2 #include <math.h>
3 #include <stdlib.h>
4
5 typedef struct Node
6 {
7     int data;
8     struct Node* prev;
9     struct Node* next;
10 } Node;
11
12 typedef struct Queue
13 {
14     Node* head;
15     Node* tail;
16 } Queue;
17
18 static void push(Queue* q, int n);
19 static void insert(Node* after, int n);
20 static void queue_dtor(Queue* queue);
21 static void print_queue(const Queue queue);
22 static Queue input_queue(void);
23 static void insert_after_odd(Queue* queue, int a1);
24
25 static Node* find_positive(Queue queue);
26 static void sort(Node* node);
27
28 static void clean_buffer(FILE* stream);
29
30 static void read_stdin(const char* save_path);
31 static double find_max_neg(const char* path);
32
33 int main()
34 {
35     puts("Лабораторная работа №5");
36
37     puts("Задание №1");
38
39     read_stdin("data.txt");
40
41     double max = find_max_neg("data.txt");
42
43     if (max == -INFINITY)
44     {
45         puts("В последовательности не было отрицательных чисел");
46     }
47     else
48     {
49         printf("max = %lg\n", max);
50     }
51
52     puts("Задание №2");
53
54     Queue q = input_queue();
55     puts("Очередь:");
56     print_queue(q);
57
58     int a1 = 0;
59     puts("Введите A1:");
60     scanf("%d", &a1);
61     insert_after_odd(&q, a1);
62     puts("Очередь:");
63     print_queue(q);
64
65     puts("Задание №3");
66     Node* pos = find_positive(q);
67     if (!pos)
68     {
69         puts("Нет положительных элементов");
70     }
71     else
72     {
73         sort(pos);
```

```

74     puts("Очередь:");
75     print_queue(q);
76 }
77
78 queue_dtor(&q);
79 }
80
81 static void push(Queue* q, int n)
82 {
83     if (q->tail == NULL)
84     {
85         q->tail = calloc(1, sizeof(Node));
86         q->tail->data = n;
87         q->head = q->tail;
88         return;
89     }
90
91     Node* new_node = calloc(1, sizeof(Node));
92     new_node->data = n;
93     new_node->prev = q->tail;
94
95     q->tail->next = new_node;
96     q->tail = new_node;
97 }
98
99 static void insert(Node* after, int n)
100 {
101     Node* new_node = calloc(1, sizeof(Node));
102     new_node->data = n;
103     new_node->next = after->next;
104     new_node->prev = after;
105
106     if (after->next)
107     {
108         after->next->prev = new_node;
109     }
110     after->next = new_node;
111 }
112
113 static void queue_dtor(Queue* queue)
114 {
115     Node* cur = queue->head;
116
117     while (cur)
118     {
119         Node* next = cur->next;
120         free(cur);
121         cur = next;
122     }
123
124     *queue = (Queue){};
125 }
126
127 static void print_queue(const Queue queue)
128 {
129     Node* cur = queue.head;
130
131     while (cur)
132     {
133         Node* next = cur->next;
134         printf("%d\n", cur->data);
135         cur = next;
136     }
137 }
138
139 static Queue input_queue(void)
140 {
141     Queue q = {};
142
143     char buffer[512] = "";
144
145     puts("Введите целые числа, ввод закончите пустой строкой:");
146
147     fgets(buffer, sizeof(buffer), stdin);
148     while (buffer[1] != '\0')
149     {

```

```

150     int n = 0;
151     sscanf(buffer, "%d", &n);
152     push(&q, n);
153     fgets(buffer, sizeof(buffer), stdin);
154 }
155
156 return q;
157 }
158
159 static void insert_after_odd(Queue* queue, int a1)
160 {
161     size_t n = 1;
162     Node* cur = queue->head;
163
164     while (cur)
165     {
166         Node* next = cur->next;
167         if (n % 2 == 1)
168         {
169             insert(cur, a1);
170         }
171         cur = next;
172         n++;
173     }
174 }
175
176 static Node* find_positive(Queue queue)
177 {
178     Node* cur = queue.head;
179
180     while (cur)
181     {
182         if (cur->data > 0)
183         {
184             return cur;
185         }
186         cur = cur->next;
187     }
188
189     return NULL;
190 }
191
192 static void sort(Node* node)
193 {
194     if (!node)
195     {
196         return;
197     }
198
199     Node* n1 = node;
200
201     while (n1)
202     {
203         Node* n2 = n1->next;
204         while (n2)
205         {
206             if (n1->data < n2->data)
207             {
208                 int t = n2->data;
209                 n2->data = n1->data;
210                 n1->data = t;
211             }
212             n2 = n2->next;
213         }
214         n1 = n1->next;
215     }
216 }
217
218 static void read_stdin(const char* save_path)
219 {
220     FILE* save = fopen(save_path, "w");
221
222     char buffer[512] = "";
223
224     puts("Введите действительные числа, ввод закончите пустой строкой:");
225

```

```

226     fgets(buffer, sizeof(buffer), stdin);
227     while (buffer[1] != '\0')
228     {
229         double n = 0;
230         sscanf(buffer, "%lg", &n);
231         fprintf(save, "%lg\n", n);
232         fgets(buffer, sizeof(buffer), stdin);
233     }
234
235     fclose(save);
236 }
237
238 static double find_max_neg(const char* path)
239 {
240     FILE* f = fopen(path, "r");
241
242     char buffer[512] = "";
243
244     double max = -INFINITY;
245
246     while (fgets(buffer, sizeof(buffer), f))
247     {
248         double n = 0;
249         sscanf(buffer, "%lg", &n);
250
251         if (n < 0 && n > max)
252         {
253             max = n;
254         }
255     }
256
257     fclose(f);
258
259     return max;
260 }
261
262 static void clean_buffer(FILE* stream)
263 {
264     while (fgetc(stream) != '\n');
265 }

```


Распечатка тестов к программе и результатов

№	Исходные данные	Результат
1	$k = 1, S[0 : 0] = [\text{abba}]$	Не нашлось подстрок, ограниченных точками Не нашлось строки со скобками и цифрами
2	$k = 2, S[0 : 2] = [\text{y..b..c ..d, .dog при(1-2)ветfe.}]$	Нашлось 3 подстрок, ограниченных точками: b c dog при(1-2)ветfe Нашлась строка со скобками и цифрами: "dog при(1-2)ветfe" После очистки букв не из русского алфавита: "привет"
3	$k = 1, S[0 : 0] = [\text{.adas.dog приветfeline.}]$	adas dog приветfeline Не нашлось строки со скобками и цифрами